

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ  
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена Трудового Красного Знамени**

**Федеральное государственное бюджетное образовательное учреждение  
высшего образования**

**«Московский технический университет связи и информатики»**

Кафедра «Математическая кибернетика и информационные технологии»

**Отчет по лабораторной работе №7**

по дисциплине «Информационные технологии и программирование»

Выполнил: студент группы

БПИ2401

Старков Дмитрий Константинович

Проверил:

Харрасов Камиль Раисович

Москва, 2025

## Содержание

|                   |    |
|-------------------|----|
| Цель работы ..... | 2  |
| Ход работы.....   | 3  |
| Вывод.....        | 10 |

## **Цель работы**

Освоить многопоточность в языке Java.

## **Ход работы**

Начнем с выполнения первого задания:

```
1 public class ArraySumMultiThread {
2
3     static class SumTask implements Runnable {
4
5         private final int[] array;
6         private final int start;
7         private final int end;
8         private long result;
9
10        public SumTask(int[] array, int start, int end) {
11            this.array = array;
12            this.start = start;
13            this.end = end;
14        }
15
16        @Override
17        public void run() {
18            long sum = 0;
19            for (int i = start; i < end; i++) {
20                sum += array[i];
21            }
22            result = sum;
23        }
24
25        public long getResult() {
26            return result;
27        }
28    }
29
30    public static void main(String[] args) throws InterruptedException {
31
32        int[] array = new int[100];
33
34        for (int i = 0; i < array.length; i++) {
35            array[i] = i + 1;
36        }
37
38        int mid = array.length / 2;
39
40        SumTask task1 = new SumTask(array, 0, mid);
41        SumTask task2 = new SumTask(array, mid, array.length);
42
43        Thread t1 = new Thread(task1);
44        Thread t2 = new Thread(task2);
45
46        t1.start();
47        t2.start();
48
49        t1.join();
50        t2.join();
51
52        long totalSum = task1.getResult() + task2.getResult();
53
54        System.out.println("Sum: " + totalSum);
55    }
56 }
57
```

В начале я создаю класс `SumTask`, который будет имплементировать `Runnable`. В этом классе я инициализирую нужные для выполнения задания поля. После чего создаю конструктор и перезаписываю метод `run()`, подстраивая его функционал под конкретную задачу подсчета элементов массива.

Далее в методе `main` я создаю сам массив и заполняю его числами от 1 до 100. Потом получаю индекс середины массива и затем создаю 2 потока – один проходит по первым 50 числам, второй – по последним. Перед запуском подсчета итоговой суммы я пишу `t1.join();` и `t2.join();` чтобы гарантированно получить результаты с дополнительных потоков.

## Теперь второе задание

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3
4 public class MatrixMaxExecutor {
5
6     static class MaxTask implements Runnable {
7
8         private final int[][] matrix;
9         private final int startRow;
10        private final int endRow;
11        private final int[] results;
12        private final int index;
13
14        public MaxTask(int[][] matrix, int startRow, int endRow, int[] results, int index) {
15            this.matrix = matrix;
16            this.startRow = startRow;
17            this.endRow = endRow;
18            this.results = results;
19            this.index = index;
20        }
21
22        @Override
23        public void run() {
24            int max = Integer.MIN_VALUE;
25            for (int i = startRow; i < endRow; i++) {
26                for (int j = 0; j < matrix[i].length; j++) {
27                    if (matrix[i][j] > max) {
28                        max = matrix[i][j];
29                    }
30                }
31            }
32            results[index] = max;
33        }
34    }
35
36    public static void main(String[] args) throws InterruptedException {
37
38        int rows = 8;
39        int cols = 6;
40        int[][] matrix = new int[rows][cols];
41
42        for (int i = 0; i < rows; i++) {
43            for (int j = 0; j < cols; j++) {
44                matrix[i][j] = i * cols + j;
45            }
46        }
47
48        int numberOfThreads = 4;
49        int rowsPerTask = rows / numberOfThreads;
50
51        ExecutorService executor = Executors.newFixedThreadPool(numberOfThreads);
52
53        int[] results = new int[numberOfThreads];
54        for (int i = 0; i < numberOfThreads; i++) {
55            int startRow = i * rowsPerTask;
56            int endRow = (i == numberOfThreads - 1) ? rows : startRow + rowsPerTask;
57
58            executor.execute(new MaxTask(matrix, startRow, endRow, results, i));
59        }
60
61        executor.shutdown();
62        while (!executor.isTerminated()) {
63            Thread.sleep(50);
64        }
65
66        int globalMax = Integer.MIN_VALUE;
67        for (int localMax : results) {
68            if (localMax > globalMax) {
69                globalMax = localMax;
70            }
71        }
72
73        System.out.println("Max: " + globalMax);
74    }
75 }
76
```



Здесь начало схожее – создание необходимых полей. В перезаписанном `run()` находится алгоритм поиска максимального элемента в матрице. Этот результат сохраняется с массив результатов с каждого потока, чтобы затем сравнить уже их и вывести глобальный максимум.

В `main` идет создание матрицы  $8 \times 6$  со значениями от 0 до 47. Потом объявление количества потоков и рядов, обрабатываемых в одном потоке. Далее идет создание пула потоков и списка результатов для каждого из них. Затем в цикле создаются и запускаются сами потоки со значениями рядов, которые были высчитаны выше. Затем `executor.shutdown()`; который закрывает пул. И финальный цикл подсчета максимального значения из локальных максимумов.

В третьем задании необходимо было написать программу для переноса груза тремя грузчиками одновременно. В моем варианте нужно было использовать `ForkJoinPool`.

```

1  import java.util.List;
2  import java.util.concurrent.ForkJoinPool;
3  import java.util.concurrent.RecursiveAction;
4
5  public class WarehouseTransfer {
6
7      static class TransferTask extends RecursiveAction {
8
9          private static final int MAX_WEIGHT = 150;
10         private final List<Integer> items;
11         private final int start;
12         private final int end;
13
14         public TransferTask(List<Integer> items, int start, int end) {
15             this.items = items;
16             this.start = start;
17             this.end = end;
18         }
19
20         @Override
21         protected void compute() {
22             int totalWeight = 0;
23             int i = start;
24             while (i < end && totalWeight + items.get(i) <= MAX_WEIGHT) {
25                 totalWeight += items.get(i);
26                 i++;
27             }
28
29             int transferredCount = i - start;
30
31             if (transferredCount == 0) {
32                 transferredCount = 1;
33             }
34
35             System.out.println("Loaders transferred " + transferredCount + " items weighing " + totalWeight + " kg.");
36
37             if (start + transferredCount < end) {
38                 TransferTask nextTask = new TransferTask(items, start + transferredCount, end);
39                 nextTask.fork();
40                 nextTask.join();
41             }
42         }
43     }
44
45     public static void main(String[] args) {
46         List<Integer> warehouse = List.of(30, 20, 50, 60, 40, 10, 80, 70, 20, 10);
47
48         ForkJoinPool pool = new ForkJoinPool(3);
49
50         TransferTask task = new TransferTask(warehouse, 0, warehouse.size());
51
52         pool.invoke(task);
53
54         pool.shutdown();
55     }
56 }
57

```

Для выполнения данного задания также необходимо было воспользоваться `RecursiveAction`, который используется для реализации методов без возвращаемого значения. Создаю класс `TransferTask`, наследующий `RecursiveAction`, а также добавляю поля списка товаров и их позиций. После чего переопределяю основной метод `RecursiveAction`, а именно `compute()`. В нем создаю переменную для общего веса и для индекса текущего товара. Затем в цикле перебираю товары до тех пор, пока их суммарный вес не превышает 150. Затем вычисляется количество перенесенных товаров, а также учитывается случай, когда `transferredCount == 0`. В этой ситуации принимается, что грузчики потратили все свои силы на перенос одного тяжелого товара. В

конце метода проверяется, достигнут ли конец списка товаров, если нет – рекурсивно создается новый поток, в котором происходит тот же алгоритм. Ну и в методе мэин создаются все необходимые классы, а затем и сами потоки. Метод `invoke()` запускает потоки и блокирует выполнение дальнейшего кода до момента, пока все задачи и подзадачи, вызванные рекурсивно, не завершатся

### **Вывод**

В ходе лабораторной работы мной была освоена многопоточность в языке Java.

Ссылка на гит - <https://github.com/BestStarProgrammer/IT-P/tree/main/lab2>